

4gaBoard Agent —QA 回答汇总

封面页: 4gaBoard Agent

3. “端到端验证”的具体标准是什么? 通过率如何定义?

端到端验证分两层: - **规则验证** (verifier.py:65): 检查最终页面 URL 是否匹配预期模式、关键元素是否存在 (通过 verification_rules.json)、页面文本是否包含关键字 - **LLM 验证** (verifier.py:115): 将执行轨迹 + 页面状态发给 LLM 综合判断

通过率 = $\text{rule_based.passed} == \text{True}$ 的场景数 $\div$ 总场景数。双重验证中任一层通过均可。

目标与核心路径页

1. 你们说“从用户手册到自动化测试”, 但用户手册本身是否可能存在不准确或过时的问题?

有的, 所以我们会参考目标页面的结构和应用源代码, 这个本身是个开源应用, 所以可以轻松获取源代码。具体通过 demo_crawler.py 启动 Playwright 登录演示站, 爬取实际 UI 元素 (按钮文本、输入框 placeholder、弹窗内容等), 与手册内容互补。 (demo_crawler.py:94 —get_demo_ui_context())

2. RAG+LLM 生成测试场景的过程中, 如何保证生成内容不重复、不遗漏关键功能?

在切分文档的时候进行精确的切分。EmbeddingRetriever 使用 RecursiveCharacterTextSplitter(chunk_size=1000, chunk_overlap=200), 按 Markdown 标题层级分隔, 保留文档结构; PageIndexRetriever 按关键词 + 标题双重打分。LLM 在每个功能点生成时检索 top-5 相关文档块作为上下文, 确保信息覆盖 (embedding.py:63, scenario_generator.py:103)。

3. Playwright + LLM 的智能体中, LLM 是如何参与决策的? 是每次执行都调用吗?

选用类似 ReACT 的架构。LLM 在三个环节参与: 1. **动态计划调整** (planner.py:49 —plan_recovery): 步骤失败时调用 LLM 生成替代操作 2. **计划优化** (planner.py:39 —_adjust_with_llm): 根据页面状态判断是否跳过/调整步骤 3. **结果验证** (verifier.py:115 —verify_with_llm): 执行完成后调用 LLM 判断场景是否通过 仅在步骤失败、执行前优化、执行后验证时调 LLM, 非每一步都调。

4. 整个流程中, 哪一部分最容易被 LLM “误导”? 你们是如何控制的?

最容易出问题的环节是**场景生成阶段 LLM 输出的 target 与实际 UI 元素不对齐** (例如生成 target=“提交” 但真实 DOM 上是“确定” 按钮)。

控制手段: 1. **多重定位降级** (executor.py:87 —_locate()): 11 种策略依次尝试 (role → label → placeholder → text → title → name → alt text → test id → CSS selector) 2. **重试机制** (agent.py:181): 每步最多重试 2 次 3. **LLM 兜底** (planner.py:49 —plan_recovery): 失败时生成替换方案 4. **双重验证**: 规则 + LLM 防止单点误判

技术选型页

1. 为什么选择 DeepSeek V4 Flash 而不是 GPT-4 或 Claude?

便宜。DeepSeek V4 Flash 定价 \$0.14/1M input + \$0.28/1M output, 远低于 GPT-4。对于高频 LLM 调用 (单次全流程约 15 次调用), 价格优势显著。

2. Qwen3-Embedding-8B 的检索效果如何? 有没有对比过其他 Embedding 模型?

效果还可以, 我们对比过 BGE-M3, 效果差不多, 个人认为主要更看重切分策略。Embedding 模型在语义检索上的差距远小于文档切分质量的影响, 因此使用 RecursiveCharacterTextSplitter 按语义边界分块比模型选择更关键。

3. ChromaDB 在你们的场景中支持多大规模的数据? 能够完成本项目。手册仅 19 页, 切分后约 100-200 个文档块, ChromaDB 处理这种规模完全无压力。

4. FastAPI + Jinja2 的组合是否足够支撑后续可能的前端复杂交互？

理论上是够的。当前 Web UI 的进度轮询、历史版本切换、LLM 费用报告等功能均通过 REST API + 前端 JS 实现，Jinja2 负责初始渲染，复杂交互完全可通过 API 驱动的 SPA 行为实现。

团队分工页

1. 你们的代码是如何集成的？有没有统一的接口规范？

有的。团队通过 Pydantic v2 模型 (models.py) 统一数据定义，抽象基类 Retriever (retrieval/base.py) 定义 build_index/retrieve 接口规范，各模块间通过 FastAPI 路由 + 标准 JSON 格式通信。

2. 如果某位同学的任务延迟，其他同学是否有备用方案？

会 push 一下。关键模块（检索器、执行器、验证器）之间有清晰的接口边界，临时调整分工可以在接口层面完成对接。

3. Web UI 和 Agent 之间的通信是同步还是异步？有没有考虑过任务队列？

异步。全流程生成使用 asyncio.to_thread 将阻塞任务放到线程池执行 (main.py:178)，前端轮询 /api/progress 获取进度。考虑过 Celery / Redis 任务队列方案，当前单机部署场景下 asyncio 已够用。

4. 你们有没有使用版本控制工具？分支管理策略是什么？

git，每个人一个分支，由韩晨旭负责审核和合入主分支。

数据流转与生命周期页

1. 你们是如何保证 Pydantic 模型与 LLM 输出的自然语言严格对齐的？

三级处理管线 (scenario_generator.py:24-43)：1. _clean_json()：去除 markdown 代码块标记 (json /) 2. _parse_json_safe()：多位置尝试解析 JSON (从第一个 [或 { 重新定位) 3. Pydantic 构造校验：TestStep(**s)、TestScenario(**item) 捕获类型/字段错误

2. 如果 LLM 输出格式错误，系统会如何处理？有没有容错机制？

容错机制三层：- JSON 清理：去除多余标记 - 多位置 JSON 查找：在文本中从第一个 JSON 起始位置重新解析 - 重试 (scenario_generator.py:123-134)：max_retries=2，发送 FEATURE_RETRY_SYSTEM 提示 LLM 重新输出。es

3. 测试报告的输出格式是什么？是否支持 CI/CD 集成？

JSON 格式 (agent.py:80-89)，包含 rule_based (各条预期的匹配详情)、llm_based (LLM 判断结论)、event_count、screenshots。保存在 testscene/task2_report_*.json。通过 /api/testscene/list 和 /api/task2/reports API 可被 CI/CD 消费。

4. 数据流转中是否有状态丢失或重复执行的风险？

- 状态丢失防护：每次 execute_step 后调用 _capture_state() (executor.py:403) 保存 URL、可见文本、截图、关键元素到 AgentMemory.history
 - 重复执行防护：agent.py:181 — _execute_with_retry() 限制最多重试 2 次，超过后调用 plan_recovery 不再重复原始步骤；Verifier 使用最终状态 + 历史事件验证，幂等安全
-

Task1 设计页

1. 你们抓取的 19 页手册是如何划分页面的？是否有结构化解析？

页面路径在 docs_scraper.py:17 —PAGE_PATHS 中硬编码 (intro、account、import-export、structure、project、board 等)。使用 BeautifulSoup 解析：获取 <h1> 标题 → 删除 <script>/<style>/<nav>/<footer>/<header> → 提取 <main>/<article> 纯文本 → 合并多余空行。每个页面作为一个独立文档单元。

2. 为什么选择持久化缓存而不是实时抓取？是否会导致手册更新不及时？

缓存到 docs_cache/pages.json，避免每次启动重复抓取（防止被限流、提升启动速度）。手册更新时，可通过 Web UI 的“重新生成测试场景”按钮或手动删除缓存目录触发重新抓取。

3. 双检索策略的切换是手动的还是自动的？有没有策略选择的判断逻辑？

手动切换，通过 .env 中 RETRIEVER_MODE 设置（可选 embedding / page_index）。factory.py:10 — create_retriever() 读取变量创建对应实例。EmbeddingRetriever 构建失败时自动回退到 PageIndexRetriever (embedding.py:83)。

4. 关键词分词检索中，如何处理同义词或拼写错误？

page_index.py:16 — _tokenize() 使用纯正则分词（保留中文和英文 >1 字符的 token），**不处理同义词或拼写错误**。这是 PageIndex 的固有局限，适合轻量无费用场景。同义词问题由 EmbeddingRetriever 的语义匹配自然解决。

Task1 中期检视页

1. “仅利用前 800 字摘要”是怎么决定的？是否考虑过滑动窗口或分块检索？

实际代码限制为 1500 字符 (scenario_generator.py:104 — content[:1500])，意图是控制 LLM 输入 token 降低延迟。分块已在 EmbeddingRetriever 中实现：RecursiveCharacterTextSplitter(chunk_size=1000, chunk_overlap=200) 按 Markdown 标题层级滑动切分，检索出的 top-5 块拼接后送入 LLM。

2. 你说“Target 与真实 UI 元素尚未完美对齐”，能否举个具体例子？

LLM 可能生成 {"action": " 点击提交", "target": " 提交"}，但实际页面中按钮文本是“确定” (<button type="submit"> 确定 </button>)，或者按钮在弹窗 (Popover / Modal) 中，常规 locator 无法直接命中。当前 _locate() 通过 get_by_text 模糊匹配 + button[type="submit"] 优先级策略缓解但不完美。

3. 场景步骤在 2-6 步之间，是否有些功能本身就需要更多步骤？你们怎么判断标准化？

Prompt 模板限制“每个场景 2~4 步” (SCENARIO_SYSTEM_TEMPLATE)。复杂功能（如完整 CRUD）拆分为多个子场景，每个子场景覆盖单一功能切面。标准化依据：一次 UI 边界操作语义（一次点击、一次填写、一次导航视为一步）。

4. 40 个测试场景中，有没有重复或冗余的场景？如何度量“高质量”？

当前有 14 个功能点、65 个场景，暂无自动化去重机制。“高质量”标准：- 步骤数 2-4 步 - target 与实际 UI 元素可匹配 - 预期结果可观察（页面跳转 / 弹窗 / 数据变化）- 规则验证通过率高

Task2 设计页

1. 9 种交互操作是否覆盖了你应用中所有可能的用户操作？

executor.py:130 — _parse_action() 支持 10+ 种操作：click、fill、select、navigate、wait、screenshot、check、uncheck、hover、scroll。覆盖看板工具核心 CRUD，但**不包含**文件上传 (<input type="file">) 和拖拽 (drag-and-drop)，这两者在 prompt 中已通过排除指令规避。

2. 多重定位降级策略具体是怎么实现的？优先级如何？

executor.py:87 — _locate() 按优先级依次尝试：1. get_by_role("button/link/textbox") 2. get_by_label(target) 3. get_by_placeholder(target) 4. get_by_text(target, exact=False) 5. [title="target"] / [name="target"] 6. get_by_alt_text(target) / get_by_test_id(target) 7. 原生 locator(target) (CSS/XPath)

点击操作额外有 _click_by_text() 按 button → a → span → div → li 标签顺序尝试，以及 button[type="submit"] 优先匹配 + Enter 键兜底。

3. LLM 如何动态自适应调整计划？是否有失败的兜底策略？

planner.py:49 —plan_recovery(): 步骤失败时, 发送当前步骤 + 页面状态给 LLM, 返回替代步骤。agent.py:181 —_execute_with_retry(): 每步最多 2 次重试, 耗尽后调用 plan_recovery, 替换步骤仍失败则中止场景。

4. 弹窗自动处理是如何判断弹窗类型的? 有没有误关闭的情况?

全局 dialog 事件监听自动 accept 所有浏览器原生弹窗 (alert/confirm/prompt)。`_dismiss_modals()` 在每次执行步骤前查找 `button[class*="close"]`。误关闭风险存在, 但加了排除条件: 弹窗在 Sidebar/Header 区域内不上操作。

Task2 中期检视页

1. 6 个英文场景中, 唯一未通过的那个是什么原因? 能否修复?

通常为涉及“添加项目”弹窗交互的场景。失败原因: 弹窗内 input 定位失败 (弹窗使用 React Portal 渲染, 不在常规 DOM 层级)。`_do_fill()` 已有 `[class*="Popup"]` input 优先匹配逻辑 (executor.py:293), 可通过提升弹窗内元素定位优先级修复。

2. “拖拽”和“复杂文件上传”为什么难攻克? 是否有替代方案?

- **拖拽**: 需要精确坐标偏移和自定义拖拽事件模拟, LLM 生成的 target 难以编码精确位置
- **文件上传**: 需要 `setInputFiles()` API 和文件路径, 场景描述无法表达文件内容

替代方案: 在 prompt 中排除拖拽和文件上传, 集中覆盖核心 CRUD 功能。

3. 批量执行和高并发测试是后续必须的吗? 你们计划如何实现?

当前 Web UI 已支持“批量执行全部场景” (`/api/task2/run-all`)。计划: 每个场景使用独立浏览器实例, 通过 `asyncio` + 线程池实现并发执行, FastAPI 异步路由管理任务队列。

4. WebUI 和 CLI 的入口是否有权限控制? 是否支持多用户?

目前无权限控制, 单用户模式。WebUI 基于 FastAPI + Jinja2 本地部署 (127.0.0.1:8000), CLI 通过 `run.py` 参数入口。多用户需要添加 session 中间件和用户认证, 当前版本聚焦于功能验证。

中枢控制台页

1. 16 个核心 API 中, 哪些是最关键的? 有没有做 API 限流或鉴权?

关键 API: `POST /scrape-and-generate` (全流程)、`GET /api/features` (数据浏览)、`POST /api/task2/run-all` (批量执行)、`GET /api/progress` (进度轮询)。**无限流或鉴权**, 仅本地部署使用。

2. 实时进度轮询的频率是多少? 会不会对服务器造成压力?

前端约 500-1000ms 轮询 `/api/progress`。数据仅从内存字典读取 (main.py:61-78), 无 IO 操作, 对服务器几乎无压力。

3. 历史场景版本切换是怎么实现的? 是否支持回滚?

`_save_features()` 每次生成时自动存档 (`testscene_YYYY_MM_DD_HH_MM_SS.json`), `/api/testscene/list` 列出所有版本, `/api/testscene/load` 加载指定版本覆盖当前 features。支持回滚到任意历史版本。

4. 费用报告是基于真实 API 调用次数还是模拟估算?

真实 API 调用。`LLMCostTracker.on_llm_end` (llm_cost_tracker.py:33) 是 LangChain BaseCallbackHandler, 每次 LLM 调用后自动捕获 token 使用量, 按 DeepSeek V4 Flash 定价实时计算。额外提供 `get_balance()` 查询 DeepSeek 账户余额。

核心模型与成本页

1. 单次场景生成 12 次 LLM 调用是否过多？有没有优化空间？

实际流程：1 次提取功能点 + 14 次场景生成 $\approx$ 15 次。优化空间：合并多个功能点批量生成以摊薄系统 prompt 开销、缓存已生成场景、增量生成（仅处理新增功能点）。

2. 输入 7k tokens、输出 12k tokens，为什么输出比输入还多？

输出包含每个场景的完整步骤数组和预期结果数组（2-4 个场景 $\times$ 2-4 步 $\times$ 2-4 条预期），JSON 字段名重复出现。系统 prompt 中的示例模板虽计入输入 token 但仅出现一次。

3. ¥0.032 的成本是否包含重试、错误恢复等额外调用？

包含。LLMCostTracker 追踪所有 on_llm_end 事件，不区分成功调用与重试调用。重试、plan_recovery 的 LLM 调用全部累加计入成本。

4. 如果大规模使用（如 1000 个场景），成本是否可控？

可控。单次场景生成约 ¥0.032，1000 个场景约 ¥32。单次执行约 ¥0.005（定位 + 验证），1000 次约 ¥5。使用 PageIndexRetriever 可降为零检索费用，仅保留 LLM 生成成本。

双检索策略评估设计页

1. 召回率是如何计算的？你们有没有人工标注的 Ground Truth？

当前无 Ground Truth 标注集。召回率的计算方式为：对每个功能点，人工检查检索出的 top-5 文档是否覆盖该功能的对应手册页面。由于手册仅 19 页，全量人工检查成本较低。

2. 场景质量的人工盲评是谁做的？样本量多少？评分一致性如何？

团队成员进行功能检查：检查每个场景的步骤是否可达、预期结果是否可观察。样本量为全部生成场景（65 个）。评分一致性通过代码审查保证。

3. 你们是否计划做 A/B 测试来验证哪种策略更好？

可切换 RETRIEVER_MODE 分别运行后对比。对比维度：target 与 UI 实际匹配率、步骤合理性、生成成本。PageIndex 零费用但缺乏语义；Embedding 检索更准但依赖 API。

4. 如果两种策略结果冲突，系统如何决策？

系统不支持同时运行两种策略。EmbeddingRetriever 在构建失败时自动 fallback 到 PageIndexRetriever，不存在主动冲突决策问题。两种检索器输出格式完全一致（{content, url, title}）。

项目进度与下一步计划页

1. “基础架构已基本完成” 是否意味着核心功能已经可用？

是。可用功能：文档缓存与爬取、双检索策略索引、功能点提取与场景生成（14 功能点 / 65 场景）、Playwright 自动化执行 + 规则/LLM 双重验证、Web UI（进度/费用/历史版本）、CLI 入口。

2. “变异测试” 具体怎么实现？是否已有初步设计？

初步思路：对页面 DOM 做微小改动（删除按钮、修改文本、改变跳转路径），运行现有测试场景，看哪些能捕获变异。捕获率越高，测试质量越好。目前无代码实现。

3. 你们如何衡量“系统鲁棒性”？有没有引入混沌工程？

间接指标：步骤执行重试成功率、LLM 输出格式错误恢复率、未捕获异常导致崩溃的频次。无正式混沌工程。

4. 下一步计划中，哪个任务风险最高？为什么？

Task 2 智能测试智能体的稳定执行风险最高。原因：依赖真实浏览器交互（网络/渲染不确定性）、LLM 动态调参不可预测（plan_recovery 可能生成不合理替代步骤）、看板应用的 SPA 路由和弹窗复杂度与 Playwright 定位策略的交互存在未知边界情况。

Q&A 页

1. 你们有没有考虑过 LLM 生成的内容可能存在偏见或幻觉？

考虑过，主要通过：- **双重约束**：规则验证提供可执行的硬性检查，LLM 仅辅助判断 - **Prompt 工程**：限制 target 必须引用 demo_ui_context 中实际爬取到的 UI 元素 - **结构化输出 + Pydantic 校验**：格式不正确的输出被捕获并触发重试

2. 如果用户手册本身写得不好，你们的系统能否自适应？

系统同时参考两个来源：用户手册（RAG 检索）和演示站 UI 爬取（demo_crawler.py）。手册质量低时，UI 爬取提供独立补充。EmbeddingRetriever 的语义检索可在手册表述模糊时匹配相关上下文片段。

3. 这个项目是否有开源计划？或者是否有论文产出的打算？

当前为课程项目，暂无开源或论文发表计划。

4. 你们认为这个系统最大的创新点是什么？

RAG + LLM 生成 → Playwright 自动执行 → 规则 + LLM 双重验证的完整闭环自动化测试流程：1. 从产品文档自动提取功能点并生成可执行测试场景 2. 生成的场景无需人工翻译即可被浏览器智能体直接执行 3. 规则引擎与 LLM 双重验证降低误判风险 4. 双检索策略兼顾成本与质量